COMP4801 Final Year Project Report

Exploring Optimization Opportunity for WhatsHap

Ng Ching Lap

UID: 3035687742

Supervisor: Professor Luo, Ruibang

Contents

Abstract

Haplotype phasing assigns heterozygous variants to their chromosome of origin and is a key step in genomic analysis. Long-read sequencing technologies (e.g., Oxford Nanopore Technologies, ONT) provide reads spanning many variants and can improve phasing contiguity. However, practical phasing performance depends on upstream alignment and variant calling, and on long-read characteristics such as non-uniform coverage, repetitive regions, and structured error patterns. WhatsHap is a widely used phasing tool for long reads; systematically understanding its end-to-end behavior under realistic conditions requires a controlled and reproducible benchmarking environment.

This project builds a reproducible benchmarking pipeline that generates a synthetic diploid reference genome, phased ground-truth variants and haplotypes, simulates ONT-like reads with configurable realism knobs, aligns reads, calls variants, phases using a vendored WhatsHap core, and evaluates outputs against truth. The pipeline supports both oracle and called variant sets to separate phaser-limited and caller-limited regimes. The implementation is validated with unit tests and an end-to-end test run, and the framework is then used for ablations and parameter sweeps to isolate failure modes and identify optimization directions for practical WhatsHap usage and long-read phasing workflows.

1. Introduction

Haplotype phasing assigns heterozygous variants to their chromosome of origin, enabling downstream analyses such as compound heterozygosity detection, allele-specific expression, and improved variant interpretation. Long-read sequencing provides reads that span multiple variants and can therefore improve both the contiguity and correctness of phased haplotypes compared to short-read data. In practice, however, phasing quality is not determined by the phasing algorithm alone: it is affected by read alignment quality, variant calling accuracy, and sequencing characteristics such as coverage bias, repetitive/duplicated regions, and structured error patterns.

WhatsHap is one of the most widely adopted phasing tools for long reads. While its algorithmic foundations are well studied, its behavior in practical end-to-end pipelines can vary significantly depending on upstream errors and data realism. To understand WhatsHap's capabilities and limitations in realistic and practical situations, it is necessary to have a benchmarking platform, where individual factors and parameters can be controlled and varied independently, benchmarking runs are efficient, reliable and reproducible.

1.1 Project goal

This final year project aims to:

1. Build a data synthesizing pipeline that generate genome data that simulates ONT long-read characteristics and errors.
2. Build a reproducible end-to-end benchmarking pipeline that simulates ONT-like long-read phasing workflows.
3. Evaluate WhatsHap's phasing capability under increasingly realistic and difficult conditions and quantify how different factors degrade performance.
4. Explore optimization directions or parameter configurations for practical WhatsHap usage.

1.2 Contributions

- An end-to-end long-read simulation and benchmarking pipeline capable of generating reference, truth variants/haplotypes, reads, alignments, variants, phased outputs, and standardized JSON/CSV reports.
- A dual evaluation approach using **oracle** (ground-truth) and **called** variant sets to separate phasing limitations from variant-calling limitations.
- Implemented realism "knobs" (e.g., duplicated regions, dropout, correlated error bursts, indel-rich truth) to isolate impacts on alignment/calling/phasing.
- Automated experiment running, aggregation, and plotting to support systematic studies and report-ready results.

1.3 Report organization

Section 2 covers background on long-read phasing, WhatsHap, and evaluation metrics. Sections 3–9 cover functional requirements, system design, implementation, validation, configuration management, quality assurance, and project management. Section 10 covers experimental results and optimization investigations, followed by discussion and conclusions in section 11 and 12.

2. Background

2.1 Long-read sequencing and ONT-like characteristics

Long-read sequencing reads can span kilobases to megabases, often covering many heterozygous variants within a single read. This enables long-range phasing and reduces ambiguity compared to short reads. ONT data, however, exhibits properties that complicate downstream analysis, including higher raw error rates than short reads, a mixture of substitution and indel errors, potential error clustering (bursts), and non-uniform coverage due to sequencing and mapping biases. Repetitive and duplicated regions further introduce ambiguity in read mapping, which can degrade both variant calling and phasing.

2.2 WhatsHap in practical pipelines (VCF + BAM → phased VCF)

In a typical long-read workflow, reads are aligned to a reference to produce BAM/CRAM, variants are called to produce a VCF containing genotypes, and a phasing tool assigns heterozygous variants to haplotypes. WhatsHap takes aligned reads (BAM) and variants (VCF) and extracts read-backed allele observations at variant positions. It then partitions heterozygous variants into phase blocks and computes a phasing solution within each block under an error-correction objective. The output is a phased VCF where heterozygous genotypes are phased (`0|1` / `1|0`) and annotated with phase set identifiers (`PS`) indicating blocks.

2.3 Why end-to-end benchmarking is necessary

Phasing performance depends on multiple stages:

- **Alignment:** mapping quality, ambiguous mappings in repeats/duplications, and alignment artifacts around indels.
- **Variant calling:** missing true variants (recall loss), spurious variants (precision loss), and genotype errors.
- **Phasing:** block fragmentation and phase inconsistencies (switch errors) among phased variants.

To understand where failures originate, this project evaluates WhatsHap under two regimes:

- **Oracle VCF:** uses ground-truth variants to isolate phasing behavior given a correct variant set.
- **Called VCF:** uses variants produced by a caller to measure realistic end-to-end performance, including caller limitations.

This separation makes it possible to distinguish “phaser-limited” regimes (phasing errors dominate) from “caller-limited” regimes (missing/incorrect variants dominate).

2.4 Metrics

This project reports metrics designed for long-read phasing:

- **Number of phase sets:** a measure of block fragmentation (fewer/larger blocks generally indicate better long-range phasing).
- **Switch error rate:** the fraction of adjacent phased heterozygous variants where predicted phase flips relative to truth.
- **Phasing rate on shared heterozygous sites:** the fraction of shared heterozygous variants that are phased.
- **Effective phased recall:** an end-to-end metric combining variant set overlap and correctness of phasing on truth heterozygous variants (used to compare oracle vs called regimes consistently).

3. Functional Requirements (FRS-style)

The functional requirements of the implemented benchamrking platform are covered in this section.

3.1 Perspective and scope

The system serves as a benchmarking platform for analysing WhatsHap's phasing capabilities in an ONT-like long-read phasing workflow. The pipeline follows this practical sequencing and phasing workflow:

Reference FASTA → reads FASTQ → alignment BAM → variants VCF → phased VCF → evaluation reports

The platform supports two evaluation frameworks:

- **Oracle (ground-truth) VCF:** isolates phasing performance by using the truth variant set.
- **Called VCF:** measures end-to-end performance including variant-calling limitations.

3.2 Users and operating environment

Primary users: developers or reserchers conducting controlled experiments for WhatsHap and generating plots and tables for analysis.

Operating environment constraints:

- Python modules are executed via `python -m ...` entrypoints.
- The end-to-end pipeline requires external tools on `PATH`:
  - `minimap2`
  - `samtools`
  - `bcftools`
- The pipeline runner checks tool availability at runtime and fails early if dependencies are missing.

3.3 Naming conventions and artifacts (prefix-based)

Most pipeline outputs are derived from a single `--prefix` argument used by `benchmark.longread_pipeline_runner`. Given `--prefix output/exp1/runA`, the system produces a consistent family of artifacts, including:

- Reference artifacts: `*.ref.fasta`, `*.ref.meta.json`.
- Truth artifacts: `*.truth.vcf`, `*.truth.vcf.gz`, `*.truth.meta.json`.
- Oracle VCF artifacts: `*.oracle.vcf`, `*.oracle.vcf.gz`.
- Simulated reads: `*.reads.fastq`, `*.reads.truth.tsv`.
- Alignment artifacts: `*.bam`, `*.bam.bai`.

- Called variants: `*.called.vcf.gz`, `*.called.vcf.gz.tbi`.
- WhatsHap outputs: `*.ws*.phased.vcf`, `*.ws*.eval.json`, `*.ws*.summary.json`.
- Run-level report: `*.pipeline.json`.

Aggregated experiment directories additionally produce:

- `aggregate.csv`
- plots under `plots/`

This prefix-based scheme is a functional requirement because it allows single-run reproducibility, experiment aggregation, and straightforward discovery of related artifacts.

3.4 Functional requirements

FR-1 Reference generation

The system shall generate a synthetic reference genome FASTA with configurable realism presets and optional duplicated segments.

Implementation mapping: implemented by `dataset.longread.reference`, invoked through `benchmark.longread_pipeline_ru` producing `*.ref.fasta` and `*.ref.meta.json`.

FR-2 Ground-truth variant and haplotype generation

The system shall generate diploid ground-truth variants and two haplotype sequences, and export both a truth VCF and haplotype FASTAs.

Implementation mapping: implemented by `dataset.longread.truth`, invoked through `benchmark.longread_pipeline_runner` producing `*.truth.vcf`, `*.truth.vcf.gz`, `*.hap1.fasta`, `*.hap2.fasta`, `*.truth.meta.json`, and `*.oracle.vcf.gz`.

FR-3 ONT-like read simulation

The system shall simulate long reads from the diploid haplotypes, producing FASTQ with quality strings and read-truth metadata.

Implementation mapping: implemented by `dataset.longread.readsim`, invoked through `benchmark.longread_pipeline_runn` producing `*.reads.fastq` and `*.reads.truth.tsv`, with support for configurable read-length, coverage, and burst-error models.

FR-4 Read alignment

The system shall align simulated reads to the reference and output a sorted, indexed BAM suitable for downstream variant calling and WhatsHap phasing.

Implementation mapping: performed by `benchmark.longread_pipeline_runner` using `minimap2` and `samtools`, producing `*.bam` and `*.bam.bai`.

FR-5 Variant calling

The system shall call variants from the aligned BAM against the reference and produce a called VCF.

Implementation mapping: performed by `benchmark.longread_pipeline_runner` using `bcftools mpileup | bcftools call`, producing `*.called.vcf.gz` and its index.

FR-6 WhatsHap phasing (vendored integration)

The system shall phase variants using WhatsHap with BAM+VCF inputs and produce phased VCF output, using the vendored WhatsHap core for controlled benchmarking.

Implementation mapping: exposed through `algorithms.cli.phase diploid-whats-bam`, implemented by `algorithms.diploid.whatshap_bam_driver`, and invoked through `benchmark.longread_pipeline_runner`, producing phased VCFs and run summaries.

FR-7 Phasing evaluation against truth

The system shall evaluate phased output against the truth VCF and output evaluation metrics focused on phasing correctness and fragmentation.

Implementation mapping: implemented by `benchmark.vcf_phase_eval`, invoked through `benchmark.longread_pipeline_runn` producing `*.eval.json` with metrics such as phase accuracy, switch error rate, phase-set count, and shared-site counts.

SNP-only policy for indels: when indels are enabled, the system shall support `--phase-snps-only` and `--eval-snps-only` so that SNP phasing evaluation remains valid despite possible indel-representation mismatches.

FR-8 End-to-end reporting and aggregation

The system shall produce a single machine-readable report per run and provide aggregation utilities for experiment directories.

Implementation mapping: `benchmark.longread_pipeline_runner` produces `*.pipeline.json`; `benchmark.aggregate_pipeli` converts many run reports into `aggregate.csv` for downstream analysis and plotting.

The pipeline report shall record:

- Parameter values used for the run.
- File paths for produced artifacts.
- Calling precision and recall when called-mode evaluation is enabled.
- Phasing evaluation metrics for oracle and/or called runs.

FR-9 Experiment driver for systematic studies

The system shall provide a driver to execute predefined experiment suites across multiple seeds, aggregate results, and generate plots.

Implementation mapping: implemented by `benchmark.experiment_driver`, which runs seeded experiment suites and produces per-experiment `aggregate.csv` files and report-ready plots.

3.5 Non-functional requirements

- **Reproducibility:** runs must be seed-controlled and record parameters in `pipeline.json`.
- **Traceability:** all plots and tables must be derived from stored machine-readable outputs (`pipeline.json`, `aggregate.csv`).
- **Maintainability:** core evaluation and aggregation logic must be testable and separated from legacy or exploratory scripts.
- **Portability:** code must run on a standard Linux environment with documented external tool dependencies.

4. System Design (SDD-style)

The design of the WhatsHap benchmarking platform, including architecture, component decomposition, interfaces, design decisions, and extensibility, are covered in this section.

4.1 Design objectives

The system is designed to satisfy five main objectives.

1. **Benchmark WhatsHap in practical workflows** Evaluate phasing in an end-to-end long-read pipeline (`FASTA → FASTQ → BAM → VCF → phased VCF`), not only on idealized matrices.
2. **Separate error sources** Support both oracle and called variant regimes so that phasing-limited behaviour can be distinguished from end-to-end caller-limited behaviour.
3. **Be reproducible and traceable** Ensure that every run is seed-controlled and produces a machine-readable report (`*.pipeline.json`) recording parameters, file paths, and metrics.
4. **Be modular and scriptable** Implement each pipeline stage as a dedicated `python -m ...` entrypoint so stages can be run independently or orchestrated through runners and experiment drivers.
5. **Support realism knobs and systematic sweeps** Expose configurable stressors such as duplicated regions, coverage dropout, burst errors, and indels so they can be varied independently in controlled experiments.

4.2 High-level pipeline structure

The system is organized as a sequential pipeline with clearly defined stage boundaries. Each stage has a single responsibility and produces artifacts that become inputs to the next stage.

4.2.1 Conceptual pipeline stages

1. **Reference synthesis** Generate a synthetic reference FASTA with optional complexity presets and duplicated segments.
2. **Truth synthesis** Generate phased truth variants, haplotypes, and an oracle VCF.
3. **Read simulation** Simulate ONT-like reads with configurable length, coverage, and error models.
4. **Alignment and preprocessing** Align reads to the reference and produce a sorted, indexed BAM.
5. **Variant calling** Produce a called VCF from the aligned BAM.
6. **Phasing** Phase oracle and/or called variants using the vendored WhatsHap core.
7. **Evaluation and reporting** Compare phased VCF output against truth VCF and measure performance metrics.

8. **Experiment orchestration** Conduct controlled experiments that systematically sweeps across realism settings and seed, then build reports for each benchmarking run and generate summary tables and plots.

4.2.2 Codebase mapping

The pipeline is implemented through three primary areas:

- `dataset/longread/` Generation of reference genome and truth files, and simulation of ONT-like long-read sequencing.
- `algorithms/` Command line interface for phasing and phasing drivers for WhatsHap.
- `benchmark/` End-to-end organization, evaluation, summarization, and experiment control.

This separation of the pipeline maintain independency and modularity of data generation, phasing, and benchmarking, while pipeline runners are still able to execute the full pipeline.

4.3 Component decomposition

4.3.1 `dataset.longread.reference`

Generates synthetic reference FASTA metadata where optional complexity presets and duplicated-region indications can be included for simulating realism.

4.3.2 `dataset.longread.truth`

Generates truth variants, haplotype FASTAs and truth metadata for performance evaluation, and oracle VCFs that allows isolation of phasing performance from variant-caller.

4.3.3 `dataset.longread.readsim`

Simulates ONT-like long-read sequencing with fully configurable realism knobs, including length distribution, coverage dropout, and correlated error bursts.

4.3.4 `benchmark.longread_pipeline_runner`

Single-run end-to-end pipeline runner. Validates reqruired external dependencies, executes the full pipeline from data generation to phasing performance evaluation and create a summarised `*.pipeline.json` report.

4.3.5 `algorithms.cli.phase`

Provides a unified CLI entrypoint for phasing backends so that phasing can be invoked in a consistent way from the runner.

4.3.6 `algorithms.diploid.whatshap_bam_driver`

Implements WhatsHap-based phasing from BAM + VCF using the vendored core. It performs readset construction, read selection, solving, PS assignment, and phased VCF writing. Reads covering fewer than two variants are excluded to satisfy WhatsHap read-selection requirements.

4.3.7 `benchmark.vcf_phase_eval`

Compares phased VCF output against truth and computes correctness, completeness, and fragmentation metrics.

4.3.8 `benchmark.aggregate_pipeline_reports`

Converts many per-run `*.pipeline.json` files into a single `aggregate.csv`, which acts as the standard interface between experiment execution and downstream plotting.

4.3.9 `benchmark.experiment_driver`

Runs predefined experiment suites across multiple seeds and parameter settings, then aggregates results and generates report-ready plots.

4.4 Data contracts and file-level interfaces

The design relies on stable file contracts between stages. The main exchanged artifact types are:

- **FASTA** for reference and haplotypes. (`*.ref.fasta`, `*.hap1.fasta`, `*.hap2.fasta`)
- **FASTQ** for simulated reads. (`*.reads.fastq`)
- **BAM** for sorted and indexed alignments. (`*.bam`, `*.bam.bai`)
- **VCF** for truth, oracle, called, and phased variant sets. (`*.truth.vcf.gz`, `*.oracle.vcf.gz`, `*.called.vcf.gz`, `*.ws*.phased.vcf`)
- **JSON** for metadata and machine-readable reports. (`*.ref.meta.json`, `*.truth.meta.json`, `*.summary.json`, `*.eval.json`, `*.pipeline.json`)

Prefix-based naming ensures that all artifacts from a run can be discovered from a single run identifier, which simplifies orchestration, aggregation, and cleanup.

4.5 Realism knobs as configurable stressors

The platform treats realism knobs as controlled stressors. Each knob is designed to approximate a practical difficulty mode and to be measurable through changes in calling metrics, phasing metrics, or both.

4.5.1 Stressor catalogue

The main stressors are:

- **Reference-level stressors**
  - reference complexity preset
  - duplicated segments
- **Truth-level stressors**
  - optional indels
  - truth-region constraints
- **Read-level stressors**
  - read length distribution
  - coverage dropout model
  - correlated error bursts
- **Runner / evaluation policy stressors**
  - oracle vs called VCF selection
  - SNP-only phasing / SNP-only evaluation when indels are present

These knobs are intentionally parameterized, stage-local, and fully recorded in `*.pipeline.json`, so that experiment results remain reproducible and traceable.

4.5.2 Measurement expectations

Increasing stressor severity is expected to affect the pipeline in different ways:

- **Caller-focused effects**
  - Lower `call_recall`.
  - Possible precision changes depending on thresholding and error severity.
- **Phaser-focused effects**
  - More phase sets under fragmentation-heavy stressors.
  - Possible switch-error increases when fewer reliable allele observations survive.
- **End-to-end effects**
  - The strongest performance degradation is in effective phased recall, which depends on both variant overlap and phasing completeness.

Realism knobs will be constructed as independently variable and testable stressors to validate these assumptions, before combining them for interaction and optimization studies.

4.6 Key design decisions

1. **Prefix-based artifact naming** To simplify discovery, cleanup and aggreation, a single prefix determines the artifact family of each run.
2. **Oracle vs called framework** To enable attribution and breakdown of phasing performance loss from variant calling and phasing, the platform separates phaser-limited behavior from end-to-end behavior.
3. **Vendored WhatsHap core** To maintain full control and understanding over benchmarking behavior and avoid dependency drift, the core algorithms of WhatsHap are vendored instead of imported.
4. **SNP-only phasing and evaluation when indels are present** To better evaluate the performance on SNP-phasing, the pipeline prevents indel representation differences from dominating phasing evaluation.
5. **Machine-readable single-run report as the source of truth** To ensure traceability and reproducibility, aggregation and plotting are driven from `*.pipeline.json` summaries rather than logs.

4.7 Extensibility

Two main incremental extension directions are supported by the platform design.

- **New realism knobs** can be implemented in the any data-generation stage, accessible through the CLI and pipeline runner.
- **Alternative phasers** can be integrated and evaluated by building drivers or adaptors under the `algorithms/` module and registering new subcommands in `algorithms.cli.phase`.

The extensibility of this platform makes the platform capable of future evaluation and analsysis beyond WhatsHap, with additional realism knobs and evaluation policies.

5. Implementation

The implementation of the benchmarking platform, including repository organization, end-to-end benchmark runner, WhatsHap integration, evaluation policies and automation, are covered in this section.

5.1 Repository structure

The repository is organized around three main concerns: data generation, phasing algorithms, and benchmarking/orchestration.

- `dataset/longread/` Implements long-read data generation:
  - `reference.py`: synthetic reference generation with presets and duplicated-region support
  - `truth.py`: diploid truth variants, haplotypes, oracle VCF, and optional indels
  - `readsim.py`: ONT-like read simulation with configurable length, dropout, and burst-error models
- `algorithms/` Implements phasing CLIs and backend drivers:
  - `cli/phase.py`: unified phasing entrypoint
  - `diploid/whatshap_bam_driver.py`: WhatsHap phasing from BAM + VCF using the vendored core
  - vendoring helper(s): ensure imports resolve to the vendored WhatsHap package
- `benchmark/` Implements orchestration, evaluation, aggregation, and experiment control:
  - `longread_pipeline_runner.py`: canonical single-run orchestrator producing `*.pipeline.json`
  - `vcf_phase_eval.py`: phased VCF evaluation against truth
  - `aggregate_pipeline_reports.py`: converts many run reports into `aggregate.csv`
  - `experiment_driver.py`: runs experiment suites across seeds and parameter settings
- `tests/` Unit and integration tests for core pipeline behaviour, evaluation, aggregation, and realism-specific logic
- `vendor/whatshap_core/` Vendored WhatsHap core used for controlled benchmarking and stable integration behaviour

This structure keeps data generation, phasing, and benchmarking concerns separate while preserving a single orchestration path for full runs.

5.2 End-to-end orchestration: `benchmark.longread_pipeline_runner`

The canonical workflow is implemented as a single-run pipeline runner:

- Entry point: `python -m benchmark.longread_pipeline_runner`

- Primary argument: `--prefix <path-prefix>`

The runner enforces two main contracts:

1. **Prefix-based artifact naming** All run outputs share a common prefix, making related artifacts easy to discover and aggregate.
2. **Machine-readable run report** Every run produces a `*.pipeline.json` file recording parameters, file outputs, and evaluation metrics.

5.2.1 Pipeline execution flow

For a run with prefix `P`, the runner performs the following high-level steps:

1. generate the synthetic reference and metadata
2. generate truth variants, haplotypes, and oracle VCF
3. simulate ONT-like reads
4. align reads and produce a sorted/indexed BAM
5. call variants to produce `P.called.vcf.gz`
6. phase oracle and/or called variants using the configured phasing backend
7. evaluate phased output against truth
8. write a unified `P.pipeline.json` report

The runner is therefore the single source of truth for how a complete pipeline instance is executed.

5.2.2 Toolchain validation and robustness

Before running stages that depend on external tools, the runner validates that the required executables are available on `PATH`. This includes:

- `minimap2`
- `samtools`
- `bcftools`

This early validation prevents partially completed runs caused by missing dependencies and improves reproducibility in scripted experiment execution.

5.3 WhatsHap integration: `diploid-whats-bam`

WhatsHap-based phasing is exposed through the unified CLI:

- `python -m algorithms.cli.phase diploid-whats-bam`

This entrypoint delegates to `algorithms.diploid.whatshap_bam_driver`, which implements phasing from BAM + VCF using the vendored WhatsHap core. The main phasing parameters surfaced through the driver are:

- `--bam`
- `--vcf`
- `--output-vcf`
- `--output-prefix`
- `--max-coverage`
- `--min-mapq`
- `--min-baseq`

5.3.1 Vendored WhatsHap core

The phasing backend uses the vendored WhatsHap core under `vendor/whatshap_core/` rather than a system-installed copy. This design keeps benchmarking behaviour controlled and reproducible, and avoids dependency drift across environments. It also makes backend behaviour easier to inspect and reason about during controlled experiments.

5.3.2 Read filtering and selection

The driver constructs the `ReadSet` consumed by the vendored core, applies phasing-side filters such as mapping-quality and base-quality thresholds, and enforces compatibility with WhatsHap's read-selection assumptions. In particular, reads covering fewer than two informative variants are excluded before read selection, since they cannot contribute to haplotype linkage.

This stage is important because the study evaluates not only final phasing output, but also how upstream evidence retention affects end-to-end performance.

5.3.3 Phased VCF output

The driver writes phased VCF output together with a summary JSON containing instrumentation, counts, and phasing-related metadata. These outputs are then consumed by the evaluation stage and folded into the final `*.pipeline.json` report.

5.4 Evaluation implementation: `benchmark.vcf_phase_eval`

Phasing evaluation is implemented by `benchmark.vcf_phase_eval`, which compares phased VCF output against truth and writes `*.eval.json`.

The evaluation focuses on three main aspects:

- **correctness**
  - phase accuracy under best block flip
  - switch error rate
- **completeness**
  - shared heterozygous recall
  - phasing rate on shared heterozygous sites
  - effective phased recall
- **fragmentation**
  - number of phase sets
  - phase-set structure implied by PS tags

This evaluator is used for both oracle and called regimes, allowing direct attribution of losses to phasing vs upstream variant recovery.

5.5 Indel-aware policy: SNP-only phasing/evaluation flags

When truth indels are enabled, the pipeline supports two important safeguarding options:

- `--phase-snps-only`
- `--eval-snps-only`

To ensure that SNP phasing metrics remain interpretable and accurate even if indel representations differ across truth, called, and predicted VCFs, these two options will restrict phasing input and/or evaluation truth to biallelic SNPs. This is crucial in evaluation SNP phasing performance by preventing indel representation mismatch between outputs from different stages to dominate the evaluation metrics.

5.6 Experiment automation and aggregation

5.6.1 Experiment driver

Systematic studies are executed through:

- `python -m benchmark.experiment_driver --outdir ... --seeds ...`

The single-run pipeline runner is repeatedly executed by the experiment runner under predefined parameter settings. Per-experiment directories directories, which contain per-seed outputs, aggregated summaries and plots, will be created by the experiment runner. This preserves one and only one canonical execution path of the pipeline by keeping experiment logic separated from the single-run pipeline runner.

5.6.2 Aggregation contract

The aggregation of phasing runs is performed by `benchmark.aggregate_pipeline_reports`, by converting many single-run level `*.pipeline.json` files into a single `aggregate.csv` summary. This CSV summary acts as a standardized interface between multiple experiment executions, and downstream comparison, plotting, and reporting. This helps improve traceability and reproducility by aggreagating from readable `*.pipeline.json` reports instead of terminal logs with much lower readability.

5.7 Realism knobs implementation and access

Realism knobs are implemented close to the stage where they conceptually belong in a sequencing or phasing pipeline. These knobs can be accessed through CLI, pipeline runner and experiment runner.

- **Reference-level knobs**
  - complexity preset
  - duplicated segments
- **Truth-level knobs**
  - optional indels
  - heterozygosity and SNP frequency
- **Read-level knobs**
  - read length model
  - coverage dropout model
  - correlated error bursts
- **Evaluation-policy knobs**
  - oracle (truth) vs called VCF
  - SNP-only phasing/evaluation flags

This staged implementation approach is crucial as it allows realism knobs to be changed independently. This enables controlled experiment sweeps while keeping results and summaries standardized for comparison and analysis.

6. Testing and Validation (STD-style)

The testing strategy and validation evidence for the platform, with the aim of achieving correctness and reproducibility of data generation, WhatsHap integration, evaluation, and end-to-end pipeline execution, will be covered by this section.

6.1 Testing objectives

The testing plan aims to validate these follow characteristics:

1. **Correctness of core representations and I/O**
    - Matrix representation and TSV/NPZ conversions.
    - Correctness of adapter(s) for converting data representations between project-specific format to WhatsHap-compatible inputs.
2. **Correctness of phasing outputs**
    - Phased output formatting.
    - Vendored WhatsHap core compatiblity.
    - Correct propagation of phase set and genotype (PS/GT) information.
3. **Correctness of evaluation metrics**
    - Block-flip-invariant phase accuracy.
    - Switch error rate.
    - Phase-set counting and shared-site metrics.
4. **End-to-end reproducibility**
    - Seed-controlled execution.
    - Deterministic prefix-based artifact naming and file generation.
    - Standardized reporting from `*.pipeline.json` to `aggregate.csv`.

6.2 Test levels and scope

A layered testing strategy is implemented:

- **Unit tests** Small deterministic components such as parsers, metric calculations, adapters, and aggregation logic are covered.
- **Integration tests** CLIs and drivers with minimal synthetic inputs and basic sanity checks are tested to verify output contracts (naming-system and file generation).
- **System / end-to-end smoke tests** The full long-read workflow is validated through the pipeline runner and experiment driver when external tools are available.

This separation is necessary because the practical long-read pipeline depends on external binaries (`minimap2`, `samtools`, `bcftools`) that may not be present in every test environment.

6.3 Test environment and dependencies

6.3.1 Python environment

- Python 3.11
- `pytest` for test execution

6.3.2 Vendored WhatsHap core

Tests that invoke the WhatsHap driver require the vendored `vendor/whatshap_core` package to be importable as `whatshap`. Where appropriate, such tests are skipped when `whatshap.core` is unavailable.

6.3.3 External toolchain

System-level long-read runs require:

- `minimap2`
- `samtools`
- `bcftools`

These are treated as system dependencies and are validated at runtime by the pipeline runner.

6.4 Implemented tests

The repository includes unit- and integration-level tests covering the main correctness-critical parts of the platform.

6.4.1 Adapter and WhatsHap-compatibility tests

`tests/test_whatshap_adapter.py` validates compatibility between the project's internal read representation and WhatsHap `ReadSet` requirements. In particular, it checks that variant indexing and allele encoding match the expected semantics used by the vendored phasing backend.

6.4.2 Evaluation and aggregation tests

Core reporting and aggregation logic is validated through tests that ensure run-level reports can be converted into standardized `aggregate.csv` outputs with the expected columns and derived metrics. This is important because downstream plots and tables depend on these machine-readable contracts rather than on log parsing.

6.4.3 Driver and phasing-output sanity tests

Integration-level checks validate that phasing drivers and CLIs produce the required output artifacts and that phased results satisfy basic structural expectations, including presence of phase-set information and well-formed summary/evaluation JSON outputs. These tests are intended to detect interface breakage early without requiring full experiment runs.

6.5 Smoke validation of the long-read pipeline

Because the full long-read workflow is more expensive and depends on external tools, it is validated as a smoke test rather than as part of the smallest unit-test layer.

Two orchestrators are used:

- **Single-run pipeline runner** `python -m benchmark.longread_pipeline_runner --prefix ...`
- **Experiment driver** `python -m benchmark.experiment_driver --outdir ... --seeds ...`

These smoke runs validate the practical path:

reference → truth → read simulation → BAM → VCF → WhatsHap phasing → evaluation → `pipeline.json`

Pass criteria include:

- Expected artifacts exist for the selected regime.
- `*.pipeline.json` is produced and parseable.
- `*.eval.json` contains required keys such as switch error and phase-set count.
- Aggregation produces a non-empty `aggregate.csv` with expected columns.

6.6 Validation of realism knobs

The realism knobs are validated using both metadata checks and metric-based sanity checks.

- **Duplications**
  - `*.ref.meta.json` must contain valid duplication coordinates.
  - Expected effect: reduced calling recall and/or increased fragmentation as duplication severity increases.
- **Coverage dropout**
  - Dropout parameters must be recorded in read metadata.
  - Expected effect: more phase sets and reduced shared-heterozygous recall in the called regime.

- **Correlated error bursts**
  - Burst parameters must be recorded in read metadata.
  - Expected effect: reduced calling quality and/or weaker phasing evidence.
- **Truth indels**
  - Indel introduction must be recorded in truth metadata.
  - When SNP-only policy is enabled, SNP phasing metrics should remain broadly stable across indel sweeps, indicating that indel representation mismatch is not corrupting evaluation.

This validation layer is important because the realism knobs are central to the benchmarking contribution of the project, not merely optional simulator features.

6.7 Coverage limitations and recommended additional tests

The current automated suite focuses mainly on matrix-mode, adapter, driver, and aggregation correctness, while the long-read toolchain workflow is validated through smoke runs that depend on external binaries.

Recommended additions include:

1. A very small long-read runner smoke test that is skipped automatically when external tools are unavailable.
2. Contract tests for `pipeline.json` and `aggregate.csv`.
3. Deterministic reference-meta integrity tests for duplication coordinates.
4. Explicit SNP-only policy regression tests under indel-enabled truth generation.

These additions would strengthen automated coverage further, but the current testing strategy is already sufficient to support the project's main claims about correctness, reproducibility, and controlled experiment execution.

7. Software Configuration Management Plan (SCMP-style)

The management of source code, experiment configurations, and generated results will be covered in this section.

7.1 Configuration items (CIs)

The primary configuration items are:

1. **Source code repository**
    - All Python modules and scripts under `dataset/`, `algorithms/`, and `benchmark/`
    - Vendored WhatsHap phasing core algorithm under `vendor/whatshap_core/`
    - Documentation under `docs/`
2. **Experiment configurations**
    - Experiment sets and parameter grids are specified in `benchmark/experiment_driver.py`
    - Single-run parameterization are specified by `benchmark/longread_pipeline_runner.py`
3. **Run artifacts and reports**
    - Prefix-scoped run artifacts (FASTA/FASTQ/BAM/VCF)
    - Machine-readable run reports: `*.pipeline.json`
    - Aggregated datasets: `aggregate.csv`
    - Plots derived from aggregates: `plots/*.png`

7.2 Version control strategy

Git is used to track changes to all code and documentation.

Roles of different branches:

- `main`: Stable and validated baseline for executing experiments and generating final results.
- `feature/*`: Isolated branches for development and validation of newly implemented features, including realism knobs, evaluation features, or phasing drivers.
- `cleanup/*`: Intermediate branches for refactoring of the codebase, removal of obsolete components, and documentation reorganization.

A change is considered to be ready for merging into the “main” branch once:

- Unit tests and integration tests are passed.
- End-to-end smoke run is successful and correct.
- Report artifact contracts are not affected.

7.3 Change traceability and controll

To ensure results are traceable to code versions:

- Experiment runs are associated with:
  - A commit hash.
  - A set of predetermined seeds.
  - A comprehensive record of parameter configuration in `*.pipeline.json`.
- All plots and tables from experiment runs that are included in the report must be derivable and reproducible from:
  - Parameter configuration `*.pipeline.json` files.

Dependency on terminal logs can be eliminated with this approach, maximizing report reproducibility.

7.4 Artifact naming and storage policy

To ensure consistency and facilitate cleanups, artifacts are generated with a well-defined prefix-based naming system.

For a run with prefix `P`:

- `P.ref.fasta`, `P.truth.vcf.gz`, `P.reads.fastq`, `P.bam`, `P.called.vcf.gz`.
- WhatsHap outputs: `P.ws*.phased.vcf`, `P.ws*.summary.json`, `P.ws*.eval.json`.
- Run report: `P.pipeline.json`.

Experiment directories are structured as:

- `output/<experiment_section>/...` containing per-seed runs and final `aggregate.csv` + `plots/`.

Artifacts are categorized as:

- **source-of-truth**: `*.pipeline.json`, `*.eval.json`, and `*.summary.json`.
- **derived**: `aggregate.csv`, visualizations.

7.5 Baseline and milestone management

Key milestones correspond to:

- Stable versions used for experiments, report figures and tables.
- Tagged commits.

7.6 Backup and recovery

To avoid data loss:

- Backups of scripts and documentations are created through remote Git repositories.
- Archives of experiment outputs used in the report are created and stored separately from the repositories.

8. Software Quality Assurance Plan (SQAP-style)

Quality assurance activites used to ensure the correctness and maintability of the platform will be covered in this section.

8.1 Quality assurance goals

The following quality properties are targeted by the quality assurance plan:

- **Correctness:** Evaluated metrics, phased outputs, and reports represent actual pipeline behavior accurately.
- **Reproducibility:** Seed and parameter configurations of experiment runs are recorded in `*.pipeline.json`.
- **Traceability:** Plots and tables in the report are generated from archived machine-readable data.
- **Maintainability:** The codebase is modularized and incremental implementation of realism knobs and experiments are supported.
- **Robustness:** Failures in external tools are detected and presented clearly.

8.2 Standards and conventions

- A consistent CLI-driven structure with `python -m <module>` entrypoints are followed.
- A prefix-based naming system is followed in output generation, ensuring deterministic file searching.
- Module-level documentation, under `docs/`, is maintained and updated regularly.

8.3 Verification activities

The following verification activities are used:

1. **Unit and integration tests using `pytest`**
    - Validation of metric computation, adapter correctness, and CLI routing.
    - Regression coverage for core evaluation logic is provided.
2. **System smoke validation**
    - Validates the full long-read workflow through:
        - `benchmark.longread_pipeline_runner` (single-run).
        - `benchmark.experiment_driver` (multi-run).
    - Existence of expected files and JSON/CSV contracts are verified.
3. **Realism knob validation**
    - Validation of each realism knob is performed through:
        - Integrity check for metadata (`*.ref.meta.json`, `*.reads.meta.json`, `*.truth.meta.json`).
        - Sanity check on output metrics.

8.4 Management of defects and prevention of regression

- Defects are addressed by:
  - Replicating with a fixed seed and saving the configuration `*.pipeline.json` of the failed run.
  - A regression test (unit test or small integration test) will be added when feasible.
- Changes and implementations are considered complete when:
  - Relevant tests are passed.
  - Reporting contract is preserved.

8.5 Documentation QA

Documentation is kept up-to-date with the codebase by:

- Runnable CLIs are referenced.
- New CLI flags are documented when new features are implemented.
- Ensuring documentation describe both oracle and called evaluation frameworks.

8.6 Limitations

- As full system validation would require validation of external tools, such as `minimap2`, `samtools`, and `bcftools`, only unit and integration tests are covered in CI-style testing.
- Some realism knobs are approximations of real genome sequencing characteristics. Validation of these realism features will focuses on correctness of implementation and expected effects, rather than biological truth.

9. Project Management Plan (PMP-style)

This project's deliverables, milestones, and risk management approach, will be covered in this section.

9.1 Deliverables

The expected final deliverables of this project includes:

1. **Implementation of the benchmarking platform**
    - End-to-end pipeline:
        - Reference → Truth/oracle → Read simulation → Alignment → Variant-calling → Phasing → Evaluation → Visualization
    - Vendored WhatsHap integration for controlled benchmarking and experimenting.
2. **Realism knob implementations**
    - Reference-level stressors, such as duplications and complexity presets.
    - Read-level stressors, such as dropout, length models, and correlated error bursts.
    - Truth-level stressors, such as indel insertion, with SNP-only policy for meaningful evaluation.
3. **Experiment infrastructure**
    - Experiment driver to facilitate systematic investigation on phasing performance across seeds, setups, and realism configurations.
    - Aggregation and visualization tools to produce report-ready figures and tables.
4. **Validation artifacts**
    - Outputs from unit tests, integration tests, and end-to-end smoke-runs.
    - `*.pipeline.json` reports and aggregated `aggregate.csv` datasets used by this report.
5. **Final report**
    - A detailed documentation of requirements, design, implementation, testing, experiments, findings, and conclusions for this project.

9.2 Work plan and milestones

A practical work plan for the second-phase (semester 2):

- **M1: Platform stabilization**
    - Ensure that the platform operates correctly on passes on target environment through smoke runs.
    - Ensure that output from the platform contain required information for report plots and tables.
- **M2: Realism knob implementation and validation**

  - Validation of each realism knob through targeted tests and sanity checks.
  - Confirm that the SNP-only policy is applied when indels are enabled.
- **M3: Experiment execution**
  - Execute the designed experiment suites.
  - Aggregate and visualize results, and explore optimization opportunity.
- **M4: Report completion**
  - Incorporation of experiment plots and tables into the final report.
  - Finalization of findings on optimization opportunities and limitations.

9.3 Risks and mitigations

Risk	Impact	Mitigation
Inconsistent availability and versioning of external tools	Inability and inconsistency in end-to-end experiments	Document external tools dependencies; Validation on targeted environment
Mismatch in indel representations leading to inaccurate evaluation	Misleading metrics	SNP-only phasing and evaluation flags; Validation with indel sweep experiment runs
Time constraints in final phase	Reduced experiment coverage; Immature optimization findings	Design experimentation plan; Prioritize critical experiments; Reduce parameter sweep size
Overfitting to synthetic data	Reduced realism	Approximate failure modes through realism knobs; Ensure randomness in experiments
Codebase complexity	Harder to maintain	Regular cleanup and refactoring; Isolate legacy scripts; Maintain consistency in `pipeline.json` schema

9.4 Tracking and reporting

The research process can be tracked through:

- Git commits and branch history documenting code modifications and feature implementations.
- Experiment outputs and `pipeline.json` reports for research traceability.
- Aggregated CSVs and plots for report integration.

10. Experimental Evaluation

10.1 Experimental methodology and common setup

10.1.1 Environment and toolchain

- OS / CPU: macOS / Apple M2 Max
- Python version: 3.12.12
- minimap2 version: 2.30-r1287
- samtools version: 1.23
- bcftools version: 1.23
- Vendored WhatsHap core path: `vendor/whatshap_core/whatshap/core.<extension>`

10.1.2 Pipeline regimes and evaluation policy

- **Oracle regime:** phase using `*.oracle.vcf.gz` to measure phaser-limited performance under correct variant sites.
- **Called regime:** phase using `*.called.vcf.gz` to measure end-to-end performance including calling limitations.
- **Indel policy:** when truth indels are enabled, use SNP-only phasing/evaluation (`phase_snps_only`, `eval_snps_only`) to avoid representation-induced distortion in SNP phasing metrics.

A custom adaptor layer was used to construct the `ReadSet` consumed by the vendored WhatsHap core. This adaptor was necessary to support oracle-vs-called benchmarking, unified JSON provenance, and controlled SNP-only phasing/evaluation under indel-containing truth sets. It is therefore part of the project's benchmarking infrastructure rather than a direct reproduction of the standard production-facing WhatsHap front-end.

10.1.3 Baseline defaults

Unless stated otherwise, experiments use the baseline defaults defined by the experiment driver.

- Reference / truth:
  - `ref_length = 80 kb`
  - `num_snps = 800`
  - `het_rate = 0.8`
  - no duplications, no dropout, no bursts, no truth indels
- Read simulation:
  - ONT-like simulation profile (`ont_profile = q20`)
  - `num_reads = 200` unless varied

  - read length range `2–6 kb`
  - `start_model = uniform` unless varied
- Calling / phasing:
  - `vcf_source = both`
  - baseline calling and phasing thresholds as defined by the experiment driver
- Seeds:
  - all reported results are aggregated across multiple seeds into `aggregate.csv`

10.1.4 Metrics and interpretation

Metrics are computed per run from `*.pipeline.json` and `*.eval.json`, then aggregated across seeds into `aggregate.csv`.

- **Calling quality**
  - `call_precision`
  - `call_recall`
  - `shared_snps`
- **Phasing completeness / usable output**
  - `called_shared_het_recall`
  - `called_phasing_rate_shared_het`
  - `oracle_effective_phased_recall`
  - `called_effective_phased_recall`
- **Phasing correctness**
  - `oracle_switch_error`
  - `called_switch_error`
  - `oracle_phase_accuracy`
  - `called_phase_accuracy`
- **Fragmentation**
  - `oracle_num_phase_sets`
  - `called_num_phase_sets`
- **Runtime**
  - `time_total_sec`
  - `called_time_*` stage breakdown fields where available

Oracle metrics indicate phaser-limited performance when correct sites are supplied. Called metrics indicate end-to-end performance after alignment, variant calling, and phasing. The gap between oracle and called effective phased recall is therefore a key attribution signal: it measures how much performance is lost because the correct heterozygous sites are not recovered and preserved into the phasing stage.

10.2 Baseline attribution study: depth scaling

Aim

Establish baseline performance curves as sequencing depth increases, and attribute performance limitations by comparing the oracle (phaser-limited) regime against the called (end-to-end) regime.

Setup

- Varied: `num_reads ∈ {50, 100, 200, 400}` (3 seeds each)
- Fixed: ONT-like simulation profile (`q20`), reference length `80 kb`, `800` truth SNPs (`het_rate = 0.8`), read length range `2–6 kb`, and the baseline calling/phasing thresholds

Figure 10.2.3 — Called effective phased recall vs sequencing depth (end-to-end performance). Called effective phased recall measures the fraction of truth heterozygous SNPs that end up both present in the called set and correctly phased (after best block flip). The curve is substantially lower than the oracle upper bound at low depth, showing that end-to-end performance is dominated early by limited variant recovery and callset overlap.

Results summary (means over seeds)

Calling (called regime):

- `call_recall`: 0.224 (50) → 0.399 (100) → 0.662 (200) → 0.877 (400)
- `call_precision`: 1.000 at all depths in this synthetic setup

Oracle phasing (phaser-limited):

- `oracle_effective_phased_recall`: 0.513 → 0.750 → 0.921 → 0.969
- `oracle_switch_error`: near 0 across all depths
- `oracle_num_phase_sets`: 10.7 → 5.7 → 1.7 → 1.0

Called phasing (end-to-end):

- `called_effective_phased_recall`: 0.040 → 0.140 → 0.426 → 0.845

- `called_shared_het_recall`: 0.129 → 0.281 → 0.590 → 0.854
- `called_phasing_rate_shared_het`: 0.355 → 0.517 → 0.808 → 0.991
- `called_switch_error`: high variance at 100–200 reads, near 0 at 400 reads
- `called_num_phase_sets`: 5.0 → 7.7 → 3.0 → 1.0

These trends are reflected by Figures 10.2.1–10.2.6, especially Figure 10.2.1 for the depth-limited rise in calling recall, Figure 10.2.3 for the large oracle-vs-called gap at low depth, and Figures 10.2.4–10.2.5 for the reduction in fragmentation with increasing coverage

Key observations

- O1 (Calling is depth-limited): Calling recall increases strongly with `num_reads` (0.224 → 0.877), while precision remains 1.0 in this controlled setup. Errors therefore appear mainly as missed sites rather than false positives.
- O2 (WhatsHap is accurate given correct sites): In the oracle regime, switch error remains near zero and phase accuracy remains near 1 even at low depth. The main limitation at 50–100 reads is incompleteness due to connectivity gaps.
- O3 (Residual high-depth loss remains caller-limited): At 400 reads, called phasing is nearly perfect on shared heterozygous sites (`called_phasing_rate_shared_het ≈ 0.99`, accuracy ≈ 1.0). The remaining gap between oracle and called effective phased recall is dominated by missing heterozygous sites in the called set (`called_shared_het_recall ≈ 0.85`).
- O4 (Mid-depth instability in the called regime): At 100–200 reads, the called regime shows occasional high switch error in individual seeds, likely due to seed sensitivity and small switch-error denominators when relatively few informative heterozygous sites are present.

Takeaway

The baseline depth sweep confirms that the pipeline behaves sensibly: calling recall increases with depth; oracle phasing becomes both accurate and contiguous; and end-to-end (called) phasing improves sharply but remains limited primarily by the overlap of correctly called heterozygous variants. At sufficiently high depth, WhatsHap phasing is essentially correct on available sites, implying that further end-to-end gains would require improved variant calling overlap rather than changes to the phasing algorithm itself.

10.3 Isolated realism stress studies

Aim

Assess how individual realism knobs affect calling recall, phasing completeness, phasing correctness, and fragmentation, while keeping the baseline setting otherwise fixed.

10.3.1 Duplicated regions

Setup

- Varied: `dup_segments ∈ {0, 1, 3, 5}`
- Fixed:
  - `dup_len = 3000`
  - `dup_min_gap = 500`
  - baseline depth `num_reads = 200`
  - no dropout, no bursts, no truth indels
  - phasing input source `vcf_source = both`

Results summary

- `call_recall`: 0.663 → 0.671 → 0.661 → 0.666
- `called_effective_phased_recall`: 0.436 → 0.473 → 0.479 → 0.372
- `called_shared_het_recall`: 0.590 → 0.599 → 0.586 → 0.593
- `called_phasing_rate_shared_het`: 0.819 → 0.836 → 0.848 → 0.778
- `called_phase_accuracy`: 0.890 → 0.920 → 0.960 → 0.802
- `called_switch_error`: 0.122 → 0.073 → 0.040 → 0.209
- `called_num_phase_sets`: 3.4 → 3.4 → 3.4 → 2.6
- `oracle_effective_phased_recall`: 0.925 → 0.932 → 0.924 → 0.918

These trends are reflected by Figures 10.3.1.1–10.3.1.5, especially Figure 10.3.1.2 for the drop in called effective phased recall only at the strongest duplication setting and Figure 10.3.1.3 for the corresponding switch-error increase.

Key observations

- O1 (Calling recall is largely unaffected by duplication): Across `dup_segments ∈ {0,1,3,5}`, calling recall remains nearly flat and call precision stays at 1.0.
- O2 (Oracle phasing remains stable): Oracle effective phased recall remains high and nearly unchanged, with near-zero oracle switch error throughout.
- O3 (Called-regime degradation appears mainly at the highest duplication level): At `dup_segments = 5`, called effective phased recall drops from `0.479` to `0.372`, while called switch error rises from `0.040` to `0.209`. `called_shared_het_recall` remains approximately constant, so the degradation is not primarily due to loss of shared heterozygous sites.
- O4 (Main loss is in phasing completeness/correctness on surviving sites): The strongest duplication setting reduces `called_phasing_rate_shared_het` and `called_phase_accuracy`, indicating that duplicated regions mainly weaken the consistency of phasing evidence in the called regime.

Takeaway Under the current parameterization, duplicated regions are a relatively mild stressor for variant recovery and for oracle phasing. Their main impact appears only at the highest duplication level, where end-to-end phasing becomes less complete and less accurate despite largely unchanged callset overlap.

10.3.2 Coverage dropout

Setup

- Varied: `dropout_fraction ∈ {0.00, 0.05, 0.10, 0.20}`
- Fixed:
  - `dropout_block_len = 1000`
  - baseline depth `num_reads = 200`
  - no duplications, no bursts, no truth indels
  - `start_model = dropout`
  - phasing input source `vcf_source = both`

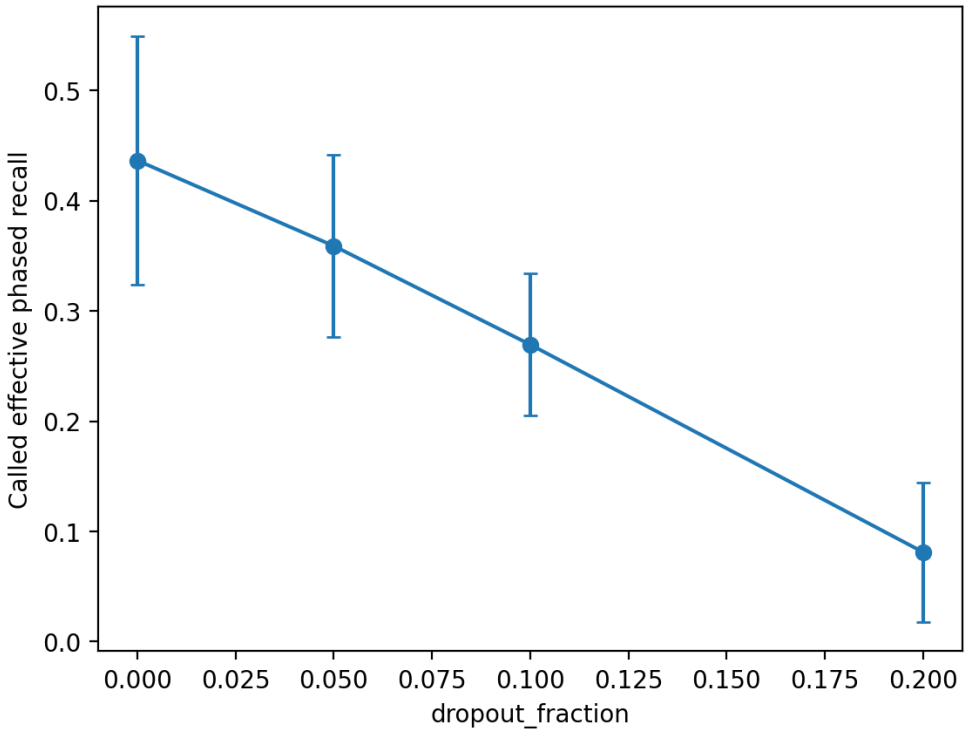

Figure 10.3.2.4 — Called effective phased recall vs dropout fraction. Called effective phased recall declines much more steeply than oracle effective phased recall, showing that dropout affects both phasing connectivity and, at stronger dropout levels, variant recovery in the called regime.

Results summary

- `call_recall`: 0.663 (0.00) → 0.635 (0.05) → 0.605 (0.10) → 0.432 (0.20)

- `oracle_effective_phased_recall`: 0.925 → 0.865 → 0.857 → 0.637
- `oracle_num_phase_sets`: 1.8 → 2.8 → 3.2 → 10.0
- `called_effective_phased_recall`: 0.436 → 0.359 → 0.269 → 0.081
- `called_shared_het_recall`: 0.590 → 0.565 → 0.529 → 0.362
- `called_phasing_rate_shared_het`: 0.819 → 0.704 → 0.514 → 0.239
- `called_num_phase_sets`: 3.4 → 5.0 → 6.2 → 4.0

These trends are reflected by Figures 10.3.2.1–10.3.2.5, especially Figure 10.3.2.4 for the steep decline in called effective phased recall and Figure 10.3.2.1 for the strong rise in oracle phase-set count.

Key observations

- O1 (Coverage dropout strongly increases fragmentation): In the oracle regime, the number of phase sets rises from 1.8 at `dropout_fraction = 0.00` to 10.0 at `0.20`, while oracle switch error remains near zero.
- O2 (Dropout directly reduces phasing completeness even with correct variant sites): Oracle effective phased recall declines from 0.925 to 0.637 as dropout increases, showing that weaker read connectivity alone can degrade phasing completeness.
- O3 (End-to-end performance degrades more strongly than oracle performance): Called effective phased recall falls from 0.436 to 0.081, showing that dropout harms end-to-end phasing through both connectivity loss and reduced overlap of correctly called heterozygous variants.
- O4 (Moderate dropout is mainly a connectivity / completeness problem): From `dropout_fraction = 0.00` to `0.10`, the main worsening in the called regime is a drop in `called_phasing_rate_shared_het` together with increased fragmentation.
- O5 (Severe dropout becomes a mixed calling + phasing bottleneck): At `dropout_fraction = 0.20`, `call_recall` drops sharply to 0.432, `called_shared_het_recall` falls to 0.362, and `called_phasing_rate_shared_het` falls to 0.239.

Takeaway Coverage dropout is the strongest isolated stressor in this study. Its primary effect is to create low-coverage gaps that prevent reads from connecting heterozygous sites into long phase blocks, reducing phasing completeness even in the oracle regime. In the called regime, this effect is amplified, and at severe dropout levels both phase connectivity and variant recovery deteriorate substantially.

10.3.3 Correlated error bursts

Results summary

- `call_recall`: 0.663 (`burst_prob = 0.0`) → 0.681 (`0.3`) → 0.676 (`0.6`)
- `oracle_effective_phased_recall`: 0.925 → 0.926 → 0.921
- `called_effective_phased_recall`: 0.436 → 0.369 → 0.455
- `called_phase_accuracy`: 0.890 → 0.766 → 0.887

- `called_switch_error`: 0.122 → 0.142 → 0.108

These trends are reflected by Figures 10.3.3.1–10.3.3.5, especially Figures 10.3.3.1 and 10.3.3.2, where the intermediate-burst dip is visible but non-monotonic.

Key observations

- Calling recall is largely unaffected, and oracle phasing remains stable.
- The main visible effect is a dip in called-regime phasing correctness at `burst_prob = 0.3`.
- The effect is not monotonic, because `burst_prob = 0.6` rebounds close to baseline on most metrics.

Takeaway Under the current parameterization, correlated error bursts are a weak and somewhat noisy called-regime correctness stressor rather than a strong practical failure mode.

10.3.4 Read length model

Results summary

- `call_recall`: 0.663 (uniform) → 0.701 (lognormal)
- `oracle_effective_phased_recall`: 0.925 → 0.931
- `oracle_num_phase_sets`: 1.8 → 1.4
- `called_effective_phased_recall`: 0.436 → 0.443
- `called_shared_het_recall`: 0.590 → 0.633
- `called_phase_accuracy`: 0.890 → 0.855
- `called_switch_error`: 0.122 → 0.165
- `called_num_phase_sets`: 3.4 → 1.6

These trends are reflected by Figures 10.3.4.1–10.3.4.5, especially Figure 10.3.4.2 for the continuity gain and Figure 10.3.4.4 for the small net change in called effective phased recall.

Key observations

- The lognormal model improves callset overlap and phase-block continuity.
- End-to-end effective phased recall changes only slightly.
- The gains in continuity and overlap are largely offset by slightly worse called-regime phasing correctness.

Takeaway The read length model is informative for mechanism analysis, but under the present parameterization it is a mixed continuity / correctness trade-off rather than a clear optimization lever.

10.3.5 Truth indels and SNP-only policy

Setup

- Varied: `num_indels ∈ {0, 80, 200}`
- Fixed:
  - `phase_snps_only = true`
  - `eval_snps_only = true`
  - baseline depth `num_reads = 200`
  - no duplications, no dropout, no bursts
  - phasing input source `vcf_source = both`

Results summary

- `call_recall`: 0.663 → 0.691 → 0.654
- `call_precision`: 1.000 → 0.999 → 0.997
- `oracle_effective_phased_recall`: 0.925 → 0.925 → 0.921
- `called_effective_phased_recall`: 0.436 → 0.573 → 0.493
- `called_shared_het_recall`: 0.590 → 0.623 → 0.577
- `called_num_phase_sets`: 3.4 → 2.4 → 4.0

These trends are reflected by Figures 10.3.5.1–10.3.5.4, which show that the main result is evaluation robustness rather than a monotonic degradation trend.

Key observations

- Oracle SNP-phasing remains stable when truth indels are introduced.
- Called SNP-phasing does not collapse in the presence of indels.
- The `80`-indel condition performs slightly better than baseline on several called metrics, but this should be interpreted as ordinary stochastic variation and indirect alignment/calling effects rather than evidence that indels improve SNP phasing.
- Even stronger indel conditions remain manageable under the SNP-only policy.

Takeaway The SNP-only policy successfully preserves interpretable SNP phasing evaluation in the presence of truth indels. This is primarily a methodology-validity result rather than a performance stress study.

10.4 Interaction study: duplicated regions × coverage dropout

Aim

Test whether duplicated regions and coverage dropout compound non-linearly, and determine whether the resulting weakness is mainly caller-limited, connectivity-limited, or mixed.

Setup

- `dup_segments ∈ {0, 5}`
- `dropout_fraction ∈ {0.0, 0.1}`
- Fixed:
  - `dropout_block_len = 1000`
  - `dup_len = 3000`
  - `dup_min_gap = 500`
  - baseline depth `num_reads = 200`
  - no bursts, no truth indels
  - phasing input source `vcf_source = both`

Results summary

- `call_recall`:
  - `dup=0, dropout=0.0`: 0.663
  - `dup=0, dropout=0.1`: 0.605
  - `dup=5, dropout=0.0`: 0.666
  - `dup=5, dropout=0.1`: 0.582
- `oracle_effective_phased_recall`:
  - 0.925 → 0.857 → 0.918 → 0.847
- `oracle_num_phase_sets`:
  - 1.8 → 3.2 → 1.8 → 4.0
- `called_effective_phased_recall`:
  - 0.436 → 0.269 → 0.373 → 0.197
- `called_shared_het_recall`:
  - 0.590 → 0.529 → 0.593 → 0.501
- `called_phasing_rate_shared_het`:
  - 0.819 → 0.514 → 0.778 → 0.410

- `called_num_phase_sets`:
  - 3.4 → 6.2 → 2.6 → 6.2

These trends are reflected by Figures 10.4.1–10.4.4, especially Figure 10.4.2 for the compounded drop in called effective phased recall and Figure 10.4.3 for the dropout-driven fragmentation pattern

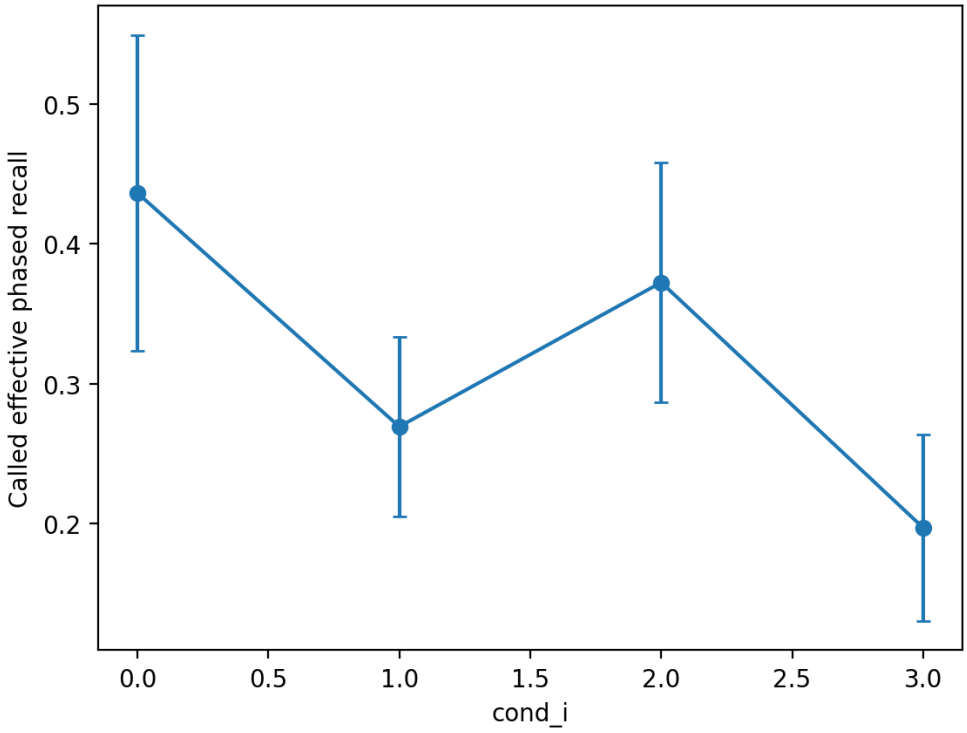

Figure 10.4.2 — Called effective phased recall across duplication × dropout conditions. The combined duplication + dropout condition produces the lowest end-to-end phased recall, showing that the two stressors interact to create a substantially more difficult phasing problem than either one alone.

Key observations

- O1 (The combined condition is the worst end-to-end setting): Called effective phased recall decreases from `0.436` in the baseline to `0.269` with dropout alone, `0.373` with duplication alone, and `0.197` when both stressors are combined.
- O2 (Dropout remains the main driver of phasing fragmentation): In the oracle regime, duplication alone has little effect, whereas dropout increases fragmentation and the combined condition increases it further.
- O3 (The interaction is strongest in the called regime): Relative to dropout alone, the combined condition further reduces `call_recall`, `called_shared_het_recall`, and `called_phasing_rate_shared_het`.
- O4 (The compounded weakness is mixed rather than purely phaser-limited): Oracle effective phased recall decreases only modestly when duplication is added on top of dropout, but called effective phased recall decreases more substantially.

Optimization implication

The duplication × dropout interaction suggests that the most important optimization opportunities under compounded stress are not limited to WhatsHap read selection. Preserving phasing evidence remains important, but upstream variant recovery in ambiguous and low-coverage regions is also a major limiting factor.

Takeaway

Duplicated regions and coverage dropout interact to produce a more severe end-to-end phasing weakness than either stressor alone. Dropout drives fragmentation, while duplication further reduces callable overlap and phasing completeness in the called regime.

10.5 Optimization under hard conditions

10.5.1 Hard-scenario setup

Optimization sweeps were conducted on a fixed composite stress condition:

- duplicated regions: `dup_segments = 5`
- coverage dropout: `dropout_fraction = 0.1`, `dropout_block_len = 1000`
- correlated error bursts: `burst_prob = 0.6`, `burst_count = 2`, `burst_len = 300`, `burst_mult = 8.0`
- truth indels: `num_indels = 120` with SNP-only phasing/evaluation enabled
- `ref_length = 120 kb`
- `num_snps = 1200`
- `num_reads = 300`

The purpose of this setting was to test whether practical parameter tuning can recover performance under combined realistic stresses rather than isolated perturbations.

10.5.2 Screening of individual knobs

Max coverage

- Increasing `max_coverage` from 10 to 40 does not improve called effective phased recall, switch error, or phase-set count, but increases runtime substantially.
- A lower-coverage sweep shows that performance plateaus by `max_coverage = 8–10`; `max_coverage = 4` is slightly too aggressive, while `15` is unnecessarily expensive.

These trends are reflected by Figures 10.5.2.1–10.5.2.4 for the coarse sweep and Figures 10.5.2.13–10.5.2.16 for the lower-coverage refinement, which together show a clear performance plateau with increasing runtime at higher retained coverage.

Phasing thresholds

- The phasing quality-threshold grid separates into two regimes:
  - `min_baseq = 0–10`: better called effective phased recall, lower switch error, fewer phase sets
  - `min_baseq = 20`: worse called and oracle phasing, more fragmentation
- `min_mapq` has negligible effect across the tested range.
- A fine sweep confirms that performance is effectively flat for `min_baseq = 0–15` and degrades only at `20`.

These trends are reflected by Figures 10.5.2.5–10.5.2.8 for the coarse phasing-threshold grid and Figures 10.5.2.17–10.5.2.20 for the fine `min_baseq` sweep, which together show that performance is stable across moderate settings and degrades only when filtering becomes too strict.

Caller thresholds

- `call_min_baseq` is the strongest caller-side optimization lever.
- Lowering `call_min_baseq` from `20` to `10` or `5` substantially improves `call_recall`, `called_shared_het_recall`, and `called_effective_phased_recall`, while call precision remains ~0.9995.
- `call_min_mapq` has only weak effects and does not provide meaningful optimization headroom.

These trends are reflected by Figures 10.5.2.9–10.5.2.12 for the caller-threshold grid and Figures 10.5.2.25–10.5.2.28 for the `call_min_mapq` rule-out sweep, showing that caller base-quality is the stronger optimization lever.

Local confirmation

- A focused local search around `call_min_baseq ∈ {5, 10, 15}` and `min_baseq ∈ {5, 10, 15}` confirms that the recommended setting lies on a stable local plateau.
- The decisive local sensitivity remains caller base-quality; phasing base-quality is effectively flat once overly strict filtering has been avoided.

These trends are reflected by Figures 10.5.2.21–10.5.2.24, which show that the recommended threshold pair lies on a stable local performance plateau.

Takeaway Optimization headroom exists, but it is uneven across knobs. The strongest useful signal is moderate caller-side base-quality filtering (`call_min_baseq = 10`), followed by avoiding overly strict phasing base-quality filtering (`min_baseq = 10`). By contrast, `min_mapq`, `call_min_mapq`, and high `max_coverage` values above the plateau are weak or negligible knobs.

10.5.3 Representative configuration comparison

Representative hard-scenario configurations:

- **`default`**
  - `call_min_mapq = 20`
  - `call_min_baseq = 15`
  - `max_coverage = 15`
  - `min_mapq = 20`
  - `min_baseq = 20`
- **`caller_only`**
  - `call_min_mapq = 20`
  - `call_min_baseq = 10`
  - `max_coverage = 15`
  - `min_mapq = 20`
  - `min_baseq = 20`
- **`phasing_only`**
  - `call_min_mapq = 20`
  - `call_min_baseq = 15`
  - `max_coverage = 8`
  - `min_mapq = 20`
  - `min_baseq = 10`
- **`optimzied`**
  - `call_min_mapq = 20`
  - `call_min_baseq = 10`
  - `max_coverage = 8`
  - `min_mapq = 20`
  - `min_baseq = 10`
- **`runtime`**
  - `call_min_mapq = 20`
  - `call_min_baseq = 10`
  - `max_coverage = 6`
  - `min_mapq = 20`
  - `min_baseq = 10`

Results summary

- `default`: `called_effective_phased_recall = 0.2919`, `called_shared_het_recall = 0.5098`, `called_num_phase_sets = 9.6`, `time_total_sec = 3.9338`
- `caller_only`: `0.2994`, `0.5181`, `9.0`, `4.0540`
- `phasing_only`: `0.2964`, `0.5098`, `6.6`, `3.7970`
- `optimzied`: `0.3041`, `0.5181`, `6.6`, `3.8682`
- `runtime`: `0.3039`, `0.5181`, `6.6`, `3.8504`

These trends are reflected by Figures 10.5.3.1–10.5.3.4, and are reinforced by the final default-vs-optimzied confirmation in Figures 10.5.3.5–10.5.3.8.

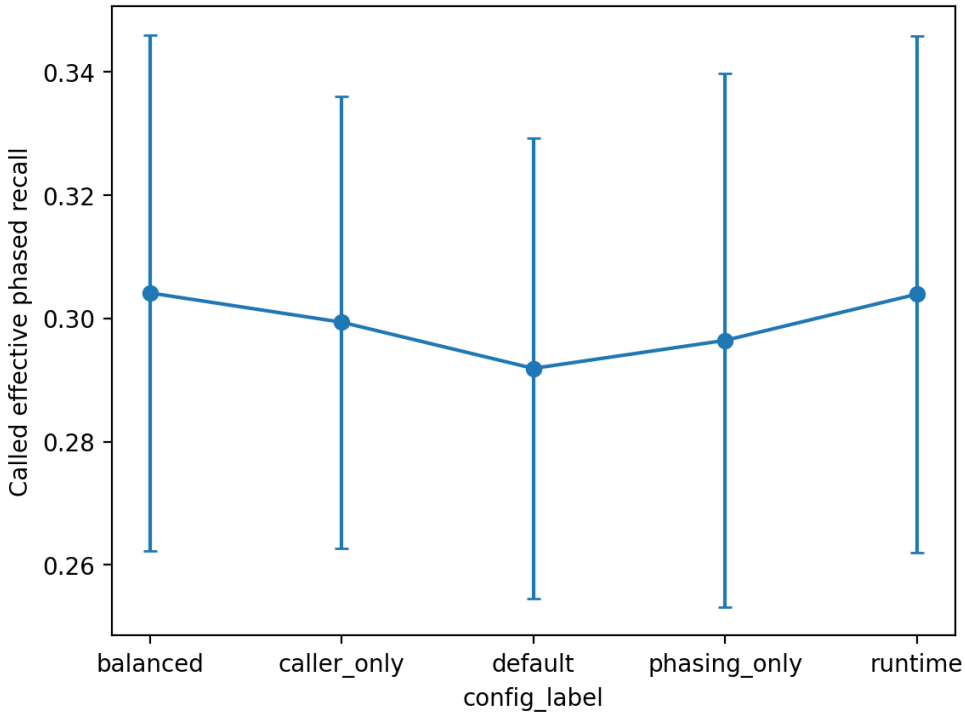

Figure 10.5.3.1 — Called effective phased recall across representative configurations. The optimzied configuration achieves the highest end-to-end phased recall, while the runtime-biased configuration performs almost identically. This shows that combining caller-side and phasing-side tuning yields the strongest practical performance.

Key observations

- O1 (The optimzied configuration provides the best overall trade-off): `optimzied` achieves the highest called effective phased recall while also maintaining low fragmentation, very low switch error, and lower runtime than the default configuration.

- O2 (Default is dominated by the tuned configurations): The default configuration is worse than `optimzied` and `runtime` on called effective phased recall, called shared heterozygous recall, switch error, and fragmentation.
- O3 (Caller-only and phasing-only tuning recover complementary parts of the gain): `caller_only` improves overlap-related metrics but leaves fragmentation high, whereas `phasing_only` strongly improves continuity and oracle phasing metrics but does not recover additional called-site overlap.
- O4 (The runtime-biased configuration is a viable alternative): `runtime` performs almost identically to `optimzied` on the main called metrics while using slightly lower retained coverage.

Takeaway

The best practical performance comes from combining moderate caller-side and phasing-side tuning rather than optimizing either stage in isolation. The optimzied configuration is the preferred general setting, while the runtime-biased configuration is a reasonable alternative when efficiency is prioritized.

10.5.4 Robustness across scenarios

Configurations compared:

- `default`
- `optimzied`
- `runtime`

Scenarios compared:

- baseline
- dropout
- interaction
- hard

Results summary

- **Baseline**
  - `default`: `called_effective_phased_recall = 0.4364`, `called_shared_het_recall = 0.5902`, `called_num_phase_sets = 3.4`
  - `optimzied`: `0.5448`, `0.6097`, `1.4`
  - `runtime`: `0.5448`, `0.6097`, `1.4`
- **Dropout**
  - `default`: `0.2694`, `0.5292`, `6.2`

  - optimzied: 0.2908, 0.5420, 4.2
  - runtime: 0.2908, 0.5420, 4.2

- **Interaction**
  - default: 0.1972, 0.5014, 6.2
  - optimzied: 0.2118, 0.5094, 5.4
  - runtime: 0.2118, 0.5094, 5.4

- **Hard**
  - default: 0.2919, 0.5098, 9.6
  - optimzied: 0.3041, 0.5181, 6.6
  - runtime: 0.3039, 0.5181, 6.6

These trends are reflected by Figures 10.5.4.1–10.5.4.4, where the tuned configurations remain above the default across all four scenarios.

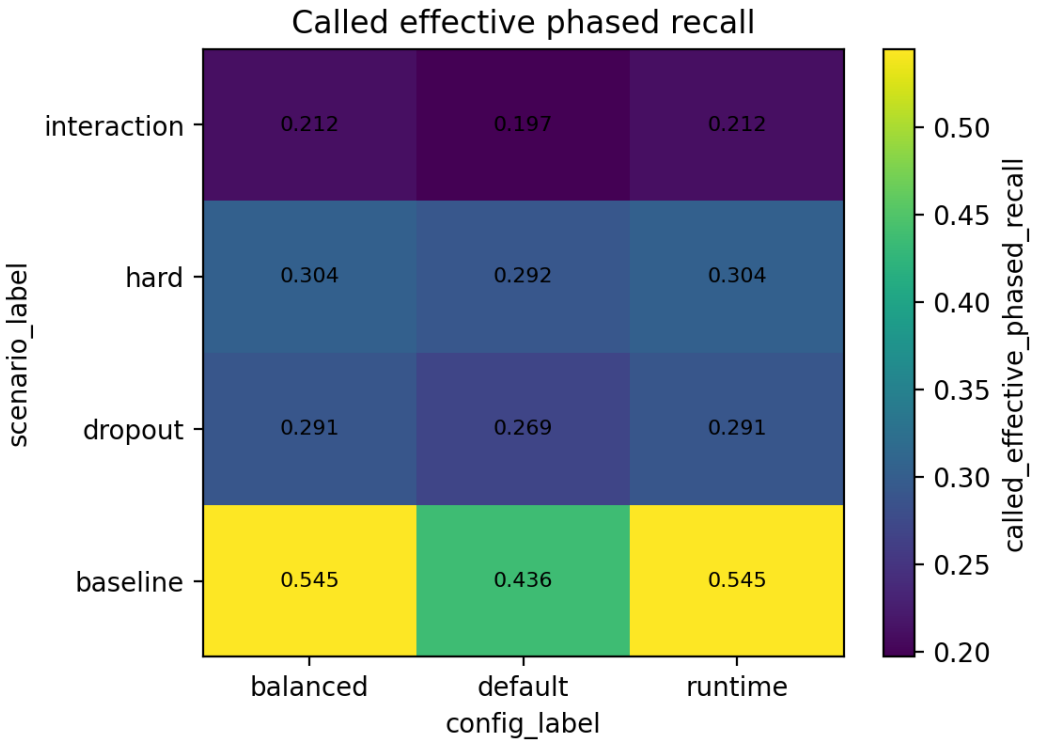

Figure 10.5.4.1 — Called effective phased recall across scenarios and representative configurations. Across all tested scenarios, the optimzied and runtime configurations achieve higher end-to-end phased recall than the default configuration, showing that the selected tuning generalizes beyond the hard optimization scenario.

Key observations

- O1 (The tuned configurations generalize across scenarios): In all four scenarios, both optimzied and runtime outperform default on called effective phased recall and called shared heterozygous recall.

- O2 (The optimzied configuration is the safest general recommendation): `optimzied` is never worse than `default` and is either best or tied for best on the main called metrics across all scenarios.
- O3 (The runtime-biased configuration is highly competitive): `runtime` is effectively identical to `optimzied` on the main called metrics and remains competitive on runtime.
- O4 (The gains transfer to both easy and hard cases): The tuned configurations improve performance not only in the hard scenario, but also in the baseline, dropout, and interaction scenarios.

Takeaway

The recommended tuned settings are robust across the representative scenarios tested in this study. The optimzied configuration is the cleanest general recommendation because it consistently improves end-to-end phased recall and reduces fragmentation relative to the default.

10.5.5 Runtime breakdown and DP-scaling interpretation

A small runtime breakdown of the called phasing stage was examined for representative hard-scenario configurations:

Configuration	Build ReadSet (s)	Read Selection (s)	Solve (s)	Called Phasing Total (s)
Default	0.3734	0.0023	0.0610	0.4433
optimzied (optimized)	0.3679	0.0061	0.0056	0.3854
Runtime (speed-oriented)	0.3685	0.0050	0.0030	0.3827

This breakdown shows that, in the current research pipeline, the largest share of called phasing runtime is spent in readset construction, while read selection and the downstream solve stage contribute smaller but configuration-dependent fractions. This result should be interpreted cautiously, because the present study uses a custom adaptor rather than the standard production-facing WhatsHap front-end.

A separate SNP-density scaling study compared `default` and `optimzied` under the same hard scenario while varying `num_snps ∈ {400, 800, 1200, 1600}`.

Results summary

- **optimized**
  - `called_time_solve_sec`: `0.0026 → 0.0046 → 0.0051 → 0.0067`
  - `oracle_time_solve_sec`: `0.0044 → 0.0066 → 0.0091 → 0.0114`

- `called_effective_phased_recall`: `0.3071 → 0.3705 → 0.3295 → 0.3280`
- `called_num_phase_sets`: `6.0 → 5.7 → 7.3 → 6.3`

- **Default**
  - `called_time_solve_sec`: `0.0053 → 0.0352 → 0.0545 → 0.1002`
  - `oracle_time_solve_sec`: `0.0114 → 0.0533 → 0.0863 → 0.1407`
  - `called_effective_phased_recall`: `0.2935 → 0.3539 → 0.3164 → 0.3141`
  - `called_num_phase_sets`: `8.3 → 10.7 → 10.0 → 7.7`

These trends are reflected by Figures 10.5.5.1–10.5.5.4, especially Figure 10.5.5.1 for the steeper default solve-time growth and Figure 10.5.5.4 for the corresponding fragmentation difference.

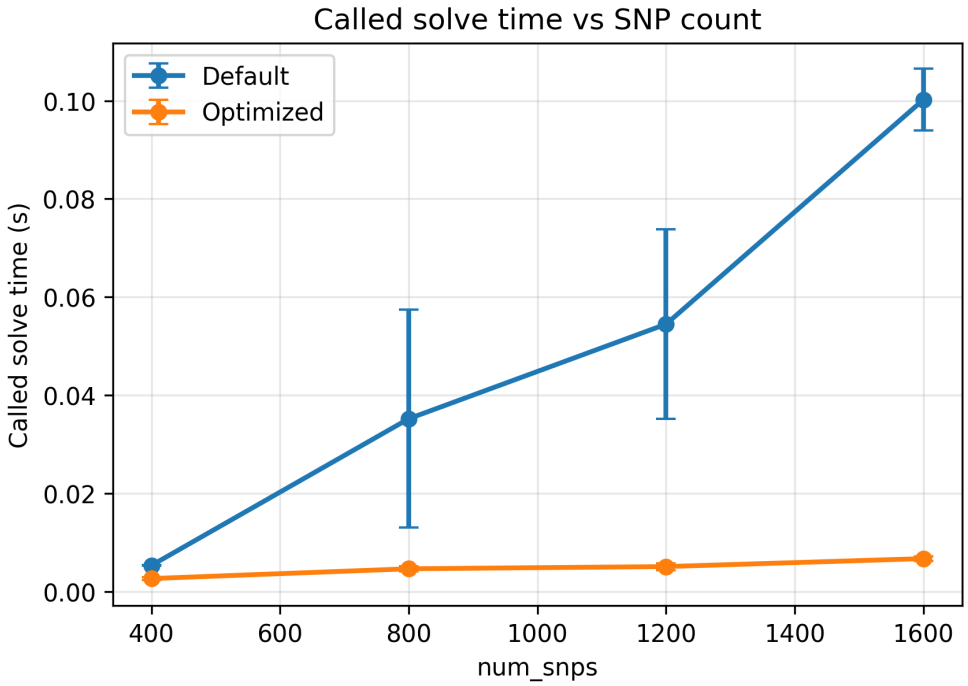

Figure 10.5.5.1 — Called solve time vs SNP count under default and optimzied configurations. Called solve time increases with SNP density under both configurations, but grows much more steeply under the default setting. This shows that the tuned configuration makes the residual phasing problem easier to solve as variant density increases.

Key observations

- O1 (Solve time increases with SNP density): In both oracle and called regimes, solve time increases as `num_snps` increases from 400 to 1600.
- O2 (The optimized configuration scales much better than default): The growth in solve time is much steeper under the default configuration than under the optimized configuration.

- O3 (Optimized remains better on phasing continuity as the problem grows): Across the scaling range, the optimized configuration generally maintains fewer phase sets and slightly better phased recall than the default configuration.
- O4 (Solve time still does not dominate total runtime end-to-end): Although solve time grows with SNP density, total phasing runtime remains dominated by preprocessing / readset construction in the current research pipeline.

Takeaway

Optimization improves not only final phased output but also the difficulty of the residual phasing problem faced by the solver. In the current research pipeline, end-to-end runtime remains dominated by preprocessing, but the tuned configuration substantially improves solve-stage scaling as SNP density increases.

10.6 Cross-experiment synthesis

10.6.1 Attribution summary (oracle vs called)

Taken together, the oracle-vs-called comparison provides a consistent attribution picture across the stress studies.

- **Duplications:** primarily a called-regime phasing / evidence-quality stressor rather than a strong caller-limited bottleneck
- **Coverage dropout:** primarily a fragmentation-dominated stressor, becoming mixed at high severity
- **Correlated error bursts:** a weak and somewhat noisy called-regime correctness stressor under the current parameterization
- **Read length model:** a mixed continuity / overlap trade-off rather than a clear optimization knob
- **Truth indels with SNP-only policy:** primarily a methodology-validity result, not a performance stressor
- **Duplication × dropout interaction:** a mixed compounded weakness, strongest in the called regime

Overall, whenever oracle performance remains strong but called performance degrades, the dominant bottleneck lies in the overlap and quality of called heterozygous sites. When oracle performance also degrades, the stressor is directly harming phase-block connectivity or phasing completeness.

10.6.2 Most impactful stressors

Across the isolated and interaction studies, the realism knobs differ substantially in practical impact.

- **Most impactful overall:** coverage dropout
- **Most impactful compounded weakness:** duplication × dropout interaction

- **Moderate impact:** strong duplication and the read-length trade-off
- **Weak / noisy impact:** correlated error bursts
- **Methodological rather than stress-inducing:** truth indels under SNP-only policy

If stressors are ranked by effect on the main end-to-end metric (`called_effective_phased_recall`), the most important findings are:

1. coverage dropout
2. duplication × dropout interaction
3. strong duplication / read-length trade-off effects
4. bursts (weak / noisy)

10.6.3 Optimization summary

Optimization sweeps under the composite hard scenario showed that optimization headroom exists, but is uneven across knobs.

- **Useful caller-side knob:** `call_min_baseq`. Lowering the caller base-quality threshold from 20 to a moderate value such as 10 substantially improves calling recall, shared heterozygous overlap, and end-to-end phased recall without materially harming precision.
- **Useful phasing-side knob:** `min_baseq`, mainly as a rule-out of overly strict filtering. Phasing performance is stable for `min_baseq = 0–15`, but degrades at `20`.
- **Useful runtime knob:** `max_coverage`. Increasing `max_coverage` beyond 8–10 does not improve phasing performance, while lower values such as 8 preserve full performance with lower runtime.
- **Weak / negligible knobs:** caller `call_min_mapq`, phasing `min_mapq`, and high `max_coverage` values above the plateau.
- **Robustness of tuning:** the tuned configurations generalize across baseline, dropout, interaction, and hard scenarios.
- **Algorithm-facing scaling result:** increasing SNP density makes the solve stage more expensive, but the optimzied tuned configuration scales much better than the default configuration.

Based on the optimization sweeps, the recommended practical settings for this pipeline are:

- **Recommended `max_coverage`:** `8`
- **Recommended phasing thresholds:** `min_mapq = 20`, `min_baseq = 10`
- **Recommended calling thresholds:** `call_min_mapq = 20` (or leave at default, as the effect is negligible), `call_min_baseq = 10`

A slightly more runtime-biased alternative (`max_coverage = 6`, with the same caller/phasing quality thresholds) performs almost identically on the main accuracy metrics and may be acceptable when efficiency is prioritized.

Final takeaway

The overall picture is consistent. End-to-end long-read phasing performance in this pipeline is limited primarily by the recovery and preservation of useful heterozygous variant evidence, not by a failure of WhatsHap to phase correctly once good sites are available. Coverage dropout is the strongest isolated realism stressor, duplication × dropout is the clearest compounded weakness, and the most useful practical optimization is to combine moderate caller-side and phasing-side thresholds rather than relying on a single WhatsHap parameter alone.

11. Discussion

11.1 Discussion of main findings

This study aimed to understand WhatsHap performance under realistic long-read conditions, identify where performance is lost in an end-to-end pipeline, and determine whether practical optimization opportunities exist. The results show that these goals were achieved at three levels: attribution, stress characterization, and optimization.

First, the baseline depth study established a clear attribution pattern. In the oracle regime, WhatsHap remained highly accurate once correct variant sites were available. In the called regime, however, a large share of end-to-end performance loss arose from reduced overlap of correctly called heterozygous variants rather than from the phasing algorithm itself. This is an important practical conclusion: in realistic settings, the phaser is not always the dominant bottleneck, and improvements in end-to-end phased recall often depend as much on variant recovery as on phasing quality.

Second, the realism-stressor studies showed that different perturbations harm the pipeline in different ways. Coverage dropout emerged as the strongest isolated stressor, primarily by fragmenting phase blocks and reducing phasing completeness, and at higher severity by also reducing variant recovery. Duplicated regions were comparatively mild under the tested parameterization, with their main effect appearing only at the strongest setting and mostly in the called regime. Correlated error bursts produced a weaker and more variable signal than expected, while the read-length model showed a mixed trade-off: better continuity and overlap, but no substantial end-to-end gain because called-regime correctness became slightly noisier. The indel study mainly served as a methodology-validation result, showing that SNP-only safeguards preserve interpretable evaluation when truth indels are present.

Third, the interaction study showed that the most informative failure modes do not necessarily come from isolated knobs alone. Duplication combined with dropout produced a substantially worse end-to-end condition than either stressor individually, especially in the called regime. This indicates that realistic phasing difficulty can arise from compounded weaknesses: ambiguous mapping worsens low-coverage conditions by simultaneously reducing callable overlap and weakening phasing completeness. This result was particularly useful because it connected the stress studies to the optimization studies: the most important practical weaknesses are mixed, rather than purely phaser-limited.

Finally, the optimization study showed that useful but limited optimization headroom exists. The strongest positive result came from caller-side base-quality tuning: reducing `call_min_baseq` from an overly strict value improved variant recovery, shared heterozygous overlap, and end-to-end phased recall without materially harming precision. On the phasing side, the clearest lesson was to avoid over-filtering: `min_baseq = 20` was consistently harmful, whereas moderate values such as `10` lay on a stable performance plateau. Retained coverage (`max_coverage`) also showed a practical runtime trade-off, with values around `8–10` preserving performance while avoiding unnecessary compute. In contrast, both caller-side and phasing-side mapping-quality thresholds provided little useful headroom.

An additional algorithm-facing result came from the SNP-density scaling study. As variant density increased, the

downstream solve stage became more expensive, but the balanced tuned configuration scaled substantially better than the default configuration. This suggests that tuning does not only improve final phasing output; it also changes the difficulty of the residual wMEC-style problem presented to the solver. Although total runtime in the research pipeline remained dominated by the project-specific readset-construction adaptor, this result still supports the view that practical tuning can improve both output quality and solver-facing problem structure.

Overall, the findings support a balanced interpretation of WhatsHap in realistic long-read pipelines. WhatsHap itself is robust when correct variant evidence is available, but end-to-end performance depends strongly on how much useful evidence survives calling, filtering, and connectivity disruptions. Optimization opportunities therefore exist, but they are fundamentally pipeline-level rather than purely phaser-internal.

11.2 Practical implications for WhatsHap-based phasing

From a practical perspective, the results suggest several concrete recommendations for WhatsHap-based long-read phasing workflows.

First, end-to-end phasing quality should not be interpreted using phasing metrics alone. In many conditions tested here, especially at moderate depth and under compounded stress, the dominant loss arose before or alongside phasing through reduced overlap of correctly called heterozygous variants. Practitioners should therefore evaluate both variant recovery and phasing quality together. A strong oracle result does not guarantee equally strong end-to-end performance if the called VCF fails to preserve enough informative heterozygous sites.

Second, conservative filtering can be counterproductive. On the calling side, overly strict base-quality filtering removed too much true variant evidence and substantially reduced downstream phased recall. On the phasing side, strict base-quality filtering similarly weakened connectivity and fragmentation metrics once the threshold became too aggressive. In the current pipeline, moderate settings consistently performed better than highly conservative ones. This supports a practical strategy of preserving useful evidence unless there is a strong reason to discard it.

Third, practical tuning should combine caller-side and phasing-side decisions. The best-performing general configuration was not obtained by optimizing the phaser in isolation, nor by optimizing the caller in isolation, but by combining both. The frontier and transferability studies further showed that this balanced tuned configuration generalizes across baseline, dropout, interaction, and hard conditions. The study therefore supports recommending a small tuned configuration set rather than a single-parameter adjustment.

Fourth, not every intuitive tuning direction is worth pursuing. Increasing `max_coverage` above the performance plateau did not help, and caller-side and phasing-side mapQ thresholds were also weak knobs. These negative findings are still useful in practice because they narrow the search space and indicate where additional tuning effort is unlikely to be rewarded.

Finally, some of the most important remaining opportunities lie upstream of the phasing solver itself. Whenever performance loss was primarily driven by missing shared heterozygous sites, improvements in variant recovery are likely to matter more than further refinement of phasing thresholds. This does not reduce the value of WhatsHap tuning, but it clarifies where future practical gains are most likely to come from.

11.3 Limitations of the present study

Several limitations should be acknowledged when interpreting the results.

First, the study uses a synthetic benchmarking pipeline rather than a real biological truth set. This was a deliberate design choice because it enabled precise control over stressors such as duplication, dropout, and correlated error bursts, and made oracle-vs-called attribution possible. However, synthetic truth still simplifies some aspects of real sequencing data and genome structure. The absolute metric values reported here should therefore be interpreted as controlled experimental measurements rather than direct predictions of production performance on all real datasets.

Second, some realism knobs produced weaker or noisier signals than expected. In particular, correlated error bursts did not show a strong monotonic degradation trend under the tested parameterization, and duplication alone was milder than expected except at the strongest setting. This does not invalidate the experiments, but it means that some conclusions are better interpreted as ruling out large effects under the current settings rather than as definitive evidence that a stressor is unimportant in general.

Third, the phasing pipeline used a custom adaptor layer to construct the `ReadSet` consumed by the vendored WhatsHap core. This adaptor was necessary for the project's experimental interface, including support for oracle/called benchmarking, unified JSON provenance, and controlled SNP-only evaluation. Runtime profiling showed that the dominant runtime cost in the research pipeline lies in this adaptor rather than in the solver itself. For this reason, the runtime results should be interpreted carefully: they are informative about the project's benchmarking infrastructure, but they do not necessarily represent the standard production-facing WhatsHap front-end directly.

Fourth, although the optimization sweeps were broad enough to identify practical recommendations, they do not exhaust the full parameter space. The recommended settings should therefore be interpreted as strong experimentally supported candidates rather than mathematically proven optima. The local search reduced this uncertainty substantially by showing that the selected threshold pair is locally robust, but broader or more adaptive searches could still be explored in future.

Fifth, the transferability and scaling studies were intentionally kept compact to remain tractable within the project scope. They are sufficient to support the main conclusions about robustness and solver-facing scaling, but they do not replace a full industrial-scale benchmarking campaign.

11.4 Future work

Several natural extensions follow from this study.

A first direction is to test the balanced tuned configuration on broader and more realistic datasets, including real long-read data or more complex synthetic references. This would help determine how far the present recommendations transfer beyond the controlled pipeline used here.

A second direction is to study upstream variant recovery more directly. Since many end-to-end losses were shown to be caller-limited, a natural extension would be to compare different variant callers, different calling models, or

more adaptive caller-side filtering strategies. The present work suggests that this may be a more promising route to further gains than continued refinement of phasing thresholds alone.

A third direction is to deepen the algorithm-facing analysis of the residual phasing problem. The SNP-density scaling study already suggests that tuning can change solver difficulty, but a richer study could investigate how variant density, read connectivity, and overlap structure jointly affect the residual wMEC problem instance presented to the DP solver.

A fourth direction is to compare the project-specific adaptor against the standard WhatsHap front-end more directly. This would help separate properties of the benchmarking interface from properties of the production-facing workflow. Such a comparison is especially relevant for runtime interpretation.

A fifth direction is to investigate region-aware optimization strategies under compounded stress. The duplication × dropout interaction suggests that ambiguous regions and low-coverage gaps may require different handling from simpler regions. Future work could therefore explore more adaptive filtering, region-aware read selection, or other strategies that respond to local evidence quality rather than using globally fixed thresholds.

11.5 Conclusion

Overall, the discussion supports a pipeline-level view of WhatsHap optimization. The experiments show that end-to-end phased recall depends strongly on upstream variant recovery, connectivity, and compounded realistic stress, while practical gains are best achieved through moderate caller-side and phasing-side filtering rather than aggressive tuning of a single parameter. These findings motivate the final conclusions summarized in Section 12.

12. Conclusion

This project investigated WhatsHap-based long-read phasing through a controlled synthetic benchmarking framework designed to balance realism, attribution, and experimental flexibility. The main objective was not only to measure phasing performance, but also to understand where performance is lost in practical end-to-end conditions and to identify optimization directions that are genuinely useful.

The project makes three main contributions. First, it establishes a realistic and configurable benchmarking pipeline that supports both oracle and called regimes. This makes it possible to separate phaser-limited behaviour from full end-to-end behaviour and to attribute losses to variant recovery, phasing completeness, or phase correctness. The framework also supports multiple realism stressors, including duplicated regions, coverage dropout, correlated error bursts, variable read-length models, and truth indels with SNP-only safeguards. This makes the platform useful not only for reporting performance, but also for studying how different conditions affect WhatsHap-based phasing mechanisms.

Second, the experiments show that end-to-end performance is often limited less by the phasing algorithm itself than by the quality and overlap of informative heterozygous variants that survive upstream processing. In the oracle regime, WhatsHap remains highly accurate once correct sites are available, while the called regime often shows larger losses due to missing shared heterozygous sites and reduced connectivity. Among the isolated realism knobs, coverage dropout was the strongest stressor, acting primarily through fragmentation and later also through reduced variant recovery. The duplication × dropout interaction further showed that realistic difficulty is often compounded rather than purely additive, with mixed caller-side and phaser-side weaknesses emerging most clearly under combined stress.

Third, the optimization study shows that useful but limited practical headroom exists. The strongest positive result came from caller-side base-quality tuning: reducing `call_min_baseq` to a moderate value improved call recall, shared heterozygous overlap, and end-to-end phased recall without materially harming precision. On the phasing side, the main lesson was to avoid overly strict filtering: `min_baseq = 20` was consistently harmful, whereas moderate values such as `10` lay on a stable performance plateau. Retained coverage also showed a practical runtime trade-off, with `max_coverage = 8` or `10` preserving performance while avoiding unnecessary compute. The recommended general setting for this pipeline is therefore `call_min_baseq = 10`, `max_coverage = 8`, and `min_baseq = 10`, with `call_min_mapq = 20` and `min_mapq = 20` retained as practical defaults. A slightly more runtime-biased alternative using `max_coverage = 6` performs almost identically on the main accuracy metrics and may also be acceptable when efficiency is prioritized.

Taken together, the results support a pipeline-level view of WhatsHap optimization. WhatsHap itself is robust when informative variant evidence is available, but end-to-end performance depends strongly on upstream variant recovery, read connectivity, and compounded realistic stress. Practical gains are therefore most likely to come from preserving useful evidence across the pipeline and from combining moderate caller-side and phasing-side tuning, rather than from aggressive tuning of a single parameter or from focusing only on the phasing solver in isolation.

Several future directions follow naturally from this work. These include validating the recommended settings on

broader and more realistic datasets, studying upstream variant recovery more directly, deepening the algorithm-facing analysis of residual solver difficulty, comparing the project-specific adaptor against the standard WhatsHap front-end, and exploring region-aware optimization strategies under compounded stress. Overall, the project provides both a reproducible evaluation platform and a practical set of experimentally supported recommendations for more effective WhatsHap-based long-read phasing.